STAT462_Assignment3

Wenjuan Wang

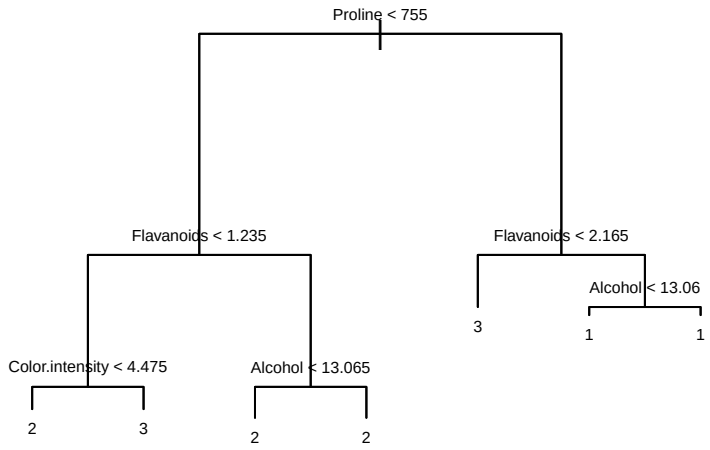
2025-10-5

Question A: Classifying wine samples (classification trees)

A.1 Train a single tree-based classifier on the training set. Use cross-validation to prune this tree suitably. Visualise the classification tree.

```
# Load the training and testing files
wine_training <- read.csv("wine_train.csv")
wine_testing <- read.csv("wine_test.csv")

# Convert the variable 'Class' into a factor variable
wine_training$Class <- as.factor(wine_training$Class)
wine_testing$Class  <- as.factor(wine_testing$Class)

# Fit a classification tree using the training data set
classification_tree <- tree(Class ~ ., data = wine_training)

# Plot the initial classification tree
plot(classification_tree)
text(classification_tree, pretty = 0, cex = 0.5)
title(main = "Figure 1. Initial Classification Tree (Before Pruning)",
      cex.main = 0.8)
```

Figure 1. Initial Classification Tree (Before Pruning)




```
# Output the summary of the  initial classification tree
summary(classification_tree)
```

```
##
## Classification tree:
## tree(formula = Class ~ ., data = wine_training)
## Variables actually used in tree construction:
## [1] "Proline"         "Flavanoids"      "Color.intensity" "Alcohol"
## Number of terminal nodes:  7
## Residual mean deviance:  0.2472 = 33.37 / 135
## Misclassification error rate: 0.05634 = 8 / 142
```

There are 7 terminal nodes in the initial classification tree, and the misclassification error rate is 5.6%.

```
# Predict on the testing data set using the initial classification tree.
test_pred <- predict(classification_tree, newdata = wine_testing, type = "class")

# Compute the misclassification error rate on test data.

pred_error_rate <- mean(test_pred != wine_testing$Class)
cat("The predicted error rate(initial tree) on the testing data set is ",
    round(pred_error_rate * 100, 2), "%\n")
```

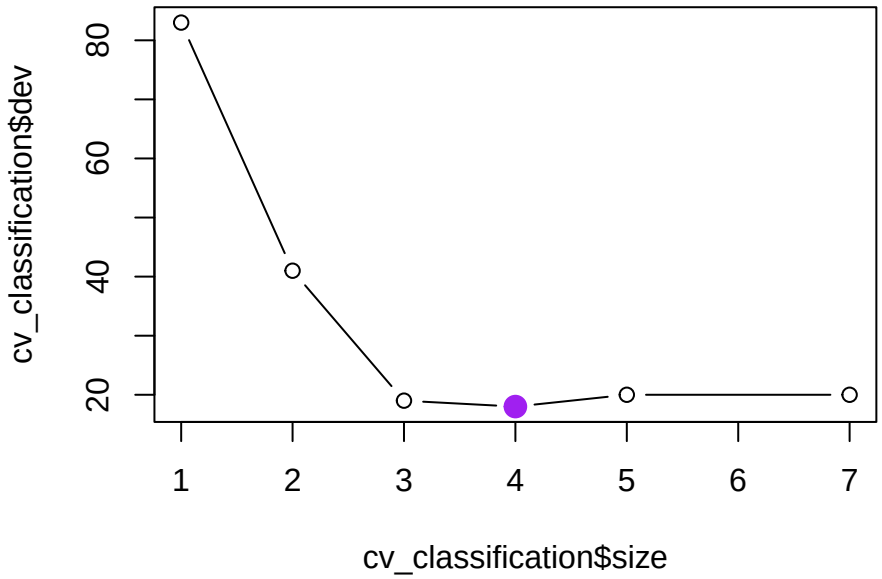
```
## The predicted error rate(initial tree) on the testing data set is  19.44 %
```

Next, we perform cross-validation in order to prune the tree and improve its generalisation performance.

```
set.seed(123)
cv_classification <- cv.tree(classification_tree, FUN = prune.misclass)
```

```r
# Plot the cross-validation results
plot(cv_classification$size, cv_classification$dev, type = "b")
title(main = "Figure 2. Cross-Validation: Deviance vs Tree Size", cex.main = 0.8)


# The optimal tree size corresponding to the lowest deviance.
best_size <- cv_classification$size[which.min(cv_classification$dev)]
best_dev  <- min(cv_classification$dev)

# Highlight the optimal tree size
points(best_size, best_dev, col = "purple", pch = 19, cex = 1.5)
```

Figure 2. Cross−Validation: Deviance vs Tree Size

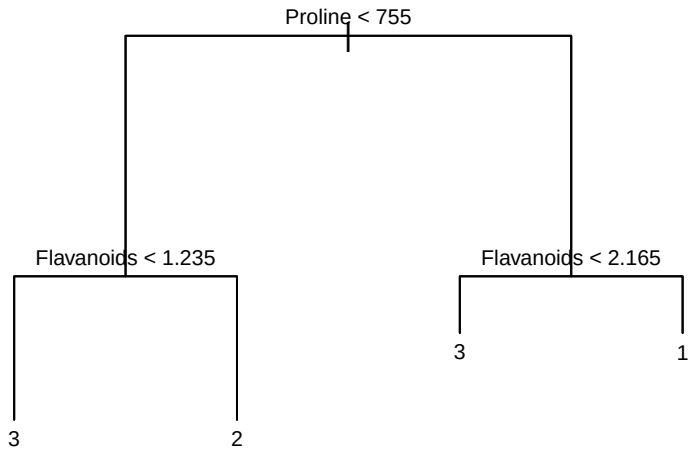



```r
cat("The optimal tree size of the pruned tree with the lowest deviance is ", best_size)
```

```
## The optimal tree size of the pruned tree with the lowest deviance is  4
```

```r
# Use the optimal tree size to prune the tree
pruned_tree <- prune.misclass(classification_tree, best = best_size)
plot(pruned_tree)
text(pruned_tree, pretty = 0, cex = 0.7)
title(main = "Figure 3. Pruned Classification Tree (After Cross-Validation)",
      cex.main = 0.8)
```

Figure 3. Pruned Classification Tree (After Cross−Validation)




```
# Make predictions on testing data using the pruned tree
pruned_test_pred <- predict(pruned_tree, newdata = wine_testing, type = "class")

# Compute the error rate
pruned_pred_error_rate <- mean(pruned_test_pred != wine_testing$Class)
cat("The predicted error rate(pruned tree) on the testing data set is ",
    round(pruned_pred_error_rate* 100, 2), "%\n")
```

```
## The predicted error rate(pruned tree) on the testing data set is  16.67 %
```

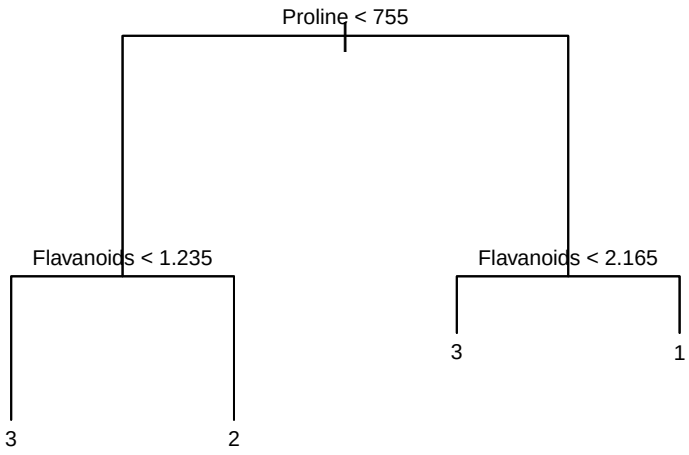
Using the training dataset, the cross-validation procedure chose the best tree size as 4 terminal nodes. Thus, pruning reduced the complexity of the tree, and the predicted error rate on the testing data decreased (from 19.44 % to 16.67%).

A.2 Prune your tree enough so that you only need two features to make predictions. Visualise your data in these two dimensions, and illustrate the decision tree of your classifier graphically.

Based on the classification tree shown in Figure 2, the tree should be pruned to a size of 4 in order to retain only two predictive features: Proline and Flavanoids.

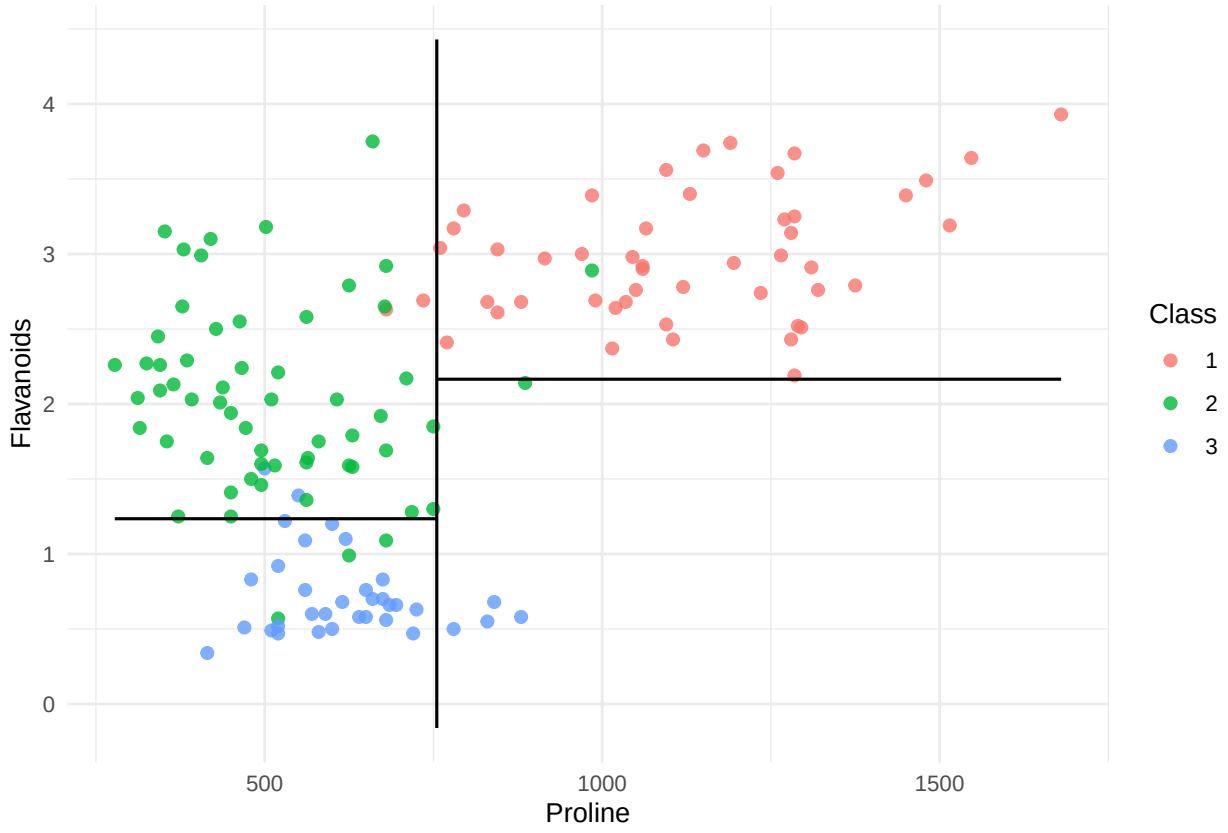
```
# Use 4 as the tree size to prune the tree to maintain only two features
pruned_tree_2feature <- prune.misclass(classification_tree, best = 4)
plot(pruned_tree_2feature)
text(pruned_tree_2feature, pretty = 0, cex = 0.7)
title(main = "Figure 4. Pruned Classification Tree (Using 2 Features)", cex.main = 0.8)
```

Figure 4. Pruned Classification Tree (Using 2 Features)




```r
# Visualize the pruned tree with two features
library(ggplot2)
df_plot <- wine_training[, c("Proline", "Flavanoids", "Class")]

xmin <- min(df_plot$Proline, na.rm = TRUE)
xmax <- max(df_plot$Proline, na.rm = TRUE)
ymin <- min(df_plot$Flavanoids-0.5 , na.rm = TRUE)
ymax <- max(df_plot$Flavanoids+0.5, na.rm = TRUE)

x_root <- 755
y_left <- 1.235
y_right <- 2.165

segs <- data.frame(
  xstart = c(x_root, xmin, x_root),
  xend   = c(x_root, x_root, xmax),
  ystart = c(ymin, y_left, y_right),
  yend   = c(ymax, y_left, y_right),
  type = c("root_v", "left_h", "right_h")
)

ggplot(df_plot, aes(x = Proline, y = Flavanoids, color = Class)) +
  geom_point(size = 2, alpha = 0.8) +
  geom_segment(data = segs, aes(x = xstart, xend = xend, y = ystart, yend = yend),
               inherit.aes = FALSE, color = "black", linewidth = 0.6) +
  theme_minimal()+
  labs(title = "Figure 5. Wine Classification: Decision Boundaries with Proline & Flavanoids")+
  theme(plot.title = element_text(hjust = 0.5, size = 12, face = "bold"))
```

Figure 5. Wine Classification: Decision Boundaries with Proline & Flavanoids



A.3 Fit a bagged classification tree model and/or a random forest to see whether you can improve on your single tree's performance.

```
# Fit the bagged classification tree model
set.seed(1)

# Count the number of predictors. Exclude Class variable.
num_predictors <- ncol(wine_training) -1

# Fit a bagged tree
bag_tree <- randomForest(Class ~ ., data = wine_training,
                         mtry = num_predictors, importance=TRUE)

# Predict on the testing set using the bagged tree.
bagged_pred <- predict(bag_tree, newdata = wine_testing)

# Compute the error rate on the testing data set
bagged_error_rate <- mean(bagged_pred != wine_testing$Class)
cat("The predicted error rate(bagged tree) on the testing data set is :",
    round(bagged_error_rate * 100, 2), "%\n")
```

```
## The predicted error rate(bagged tree) on the testing data set is : 0 %
```

```
# Fit the random forests
set.seed(1)
```

```r
# Fit a random_forests using two features
random_forests <- randomForest(Class ~ ., data = wine_training, mtry = 2, importance=TRUE)

# Predict on the testing set using the random_forests
random_forests_pred <- predict(random_forests, newdata = wine_testing)

# Compute the error rate on the testing data set
random_forests_error_rate <- mean(random_forests_pred != wine_testing$Class)
cat("The predicted error rate(random_forests) on the testing data set is :",
    round(random_forests_error_rate * 100, 2), "%\n")
```

```
## The predicted error rate(random_forests) on the testing data set is : 0 %
```

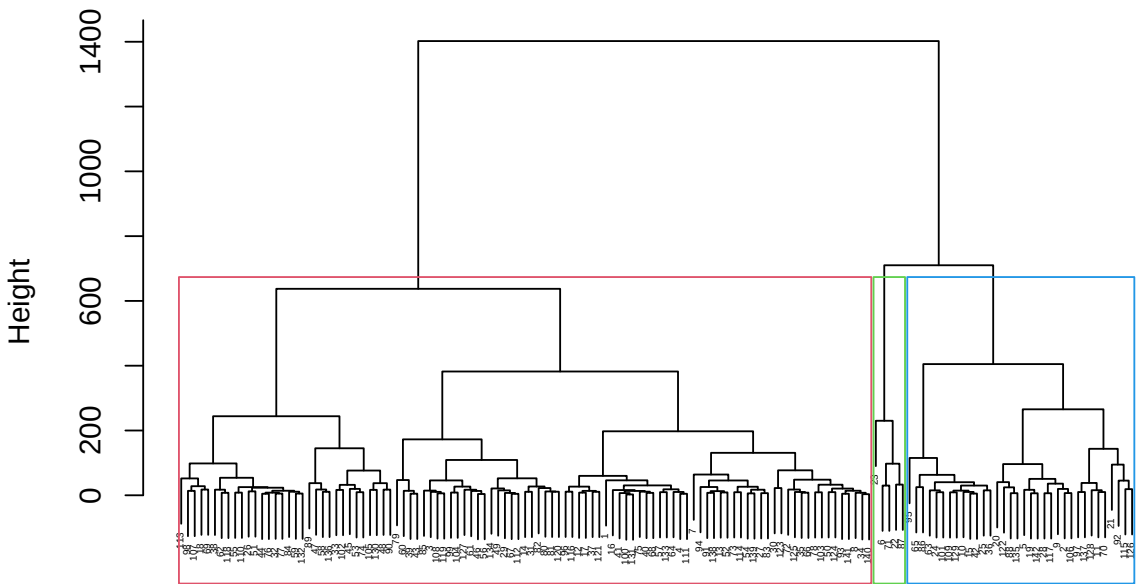
Using the bagged classification tree and random forests, the predicted error rate on the testing data set drops to 0, indicating that these ensemble methods substantially outperform the single classification tree (both the initial(19.44%) and pruned versions(19.44% and 16.67%)).

Question B: Clustering the wine dataset (Hierarchical clustering and k-means)

B.1 Perform hierarchical clustering on the wine dataset, but do not include the Class feature. Group the data into three clusters and check/visualise whether this is a good reconstruction of the (actual) classes recorded in the Class feature.

```r
# Load the wine csv files(training)
wine_training <- read.csv("wine_train.csv")

# Exclude the 'Class'
wine_training_noClass <- wine_training[, !(names(wine_training) == "Class")]

# Compute the distance matrix without using the variable 'Class'
dist_matrix <- dist(wine_training_noClass)

# perform hierarchical clustering using the complete linkage
hc_complete <- hclust(dist_matrix, method = "complete")
plot(hc_complete, main = "Figure 6. Hierarchical Clustering Using Unscaled Data",
     xlab = "", sub = "", cex = .3)

# Specify a chosen merging height
rect.hclust(hc_complete, k = 3, border = 2:4)
```

Figure 6. Hierarchical Clustering Using Unscaled Data




```
# Check the outputs of hierarchical clustering with the actual classes
# Cut the hierarchical clustering result into 3 clusters.
hc_clusters <- cutree(hc_complete, k = 3)

# Get the actual classes in the training data
actual_class <- as.factor(wine_training$Class)

# Compute the confusion matrix on training data set
cm <- table(Cluster = hc_clusters, Class = actual_class)
print(cm)
```

```
##         Class
## Cluster  1  2  3
##       1 11 58 34
##       2 33  1  0
##       3  5  0  0
```

```
# Compute the purity of the confusion matrix
purity <- sum(apply(cm, 1, max)) / sum(cm)
cat("The purity of the confusion matrix using the unscaled training data is",
    round(purity,2))
```

```
## The purity of the confusion matrix using the unscaled training data is 0.68
```

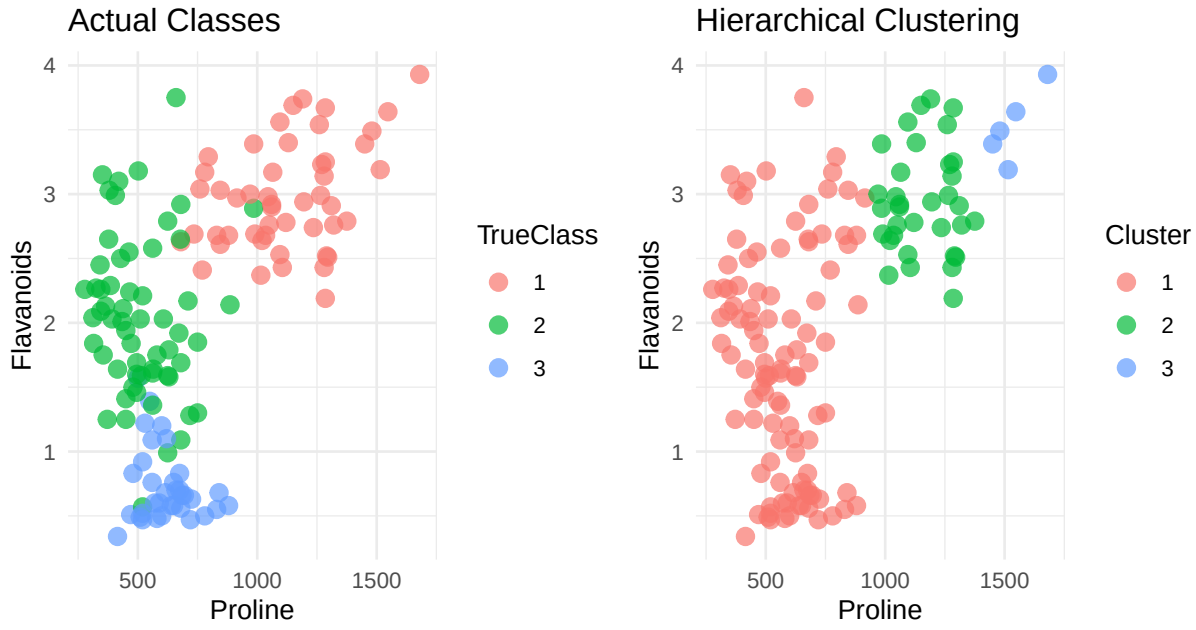
The confusion matrix shows unsatisfactory reconstruction, with the purity of the confusion matrix being 0.68. During this process, no standardization was performed; therefore, variables with larger absolute magnitudes exerted a dominant effect on the clustering outcome.

```r
# Visualise the actual classes compared to the hierarchical clustering results.
library(patchwork)
```

```
## Warning: package 'patchwork' was built under R version 4.4.3
```

```r
plot_data <- data.frame(
  Proline = wine_training$Proline,
  Flavanoids = wine_training$Flavanoids,
  TrueClass = as.factor(wine_training$Class),
  Cluster = as.factor(hc_clusters)
)

# Ground Truth
p1 <- ggplot(plot_data, aes(x = Proline, y = Flavanoids, color = TrueClass)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(title = "Actual Classes",
       x = "Proline", y = "Flavanoids") +
  theme_minimal()

# The clustering results
p2 <- ggplot(plot_data, aes(x = Proline, y = Flavanoids, color = Cluster)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(title = "Hierarchical Clustering",
       x = "Proline", y = "Flavanoids") +
  theme_minimal()

final_plot <- p1 + p2 + plot_annotation(
  title = "Figure 7. Comparison of Actual Classes vs Hierarchical Clustering",
  theme = theme(plot.title = element_text(hjust = 0.5, size = 12, face = "bold"))
)

print(final_plot)
```

Figure 7. Comparison of Actual Classes vs Hierarchical Clustering



The comparison between the clustering results and the actual classes in the testing dataset demonstrates that the reconstruction quality is unsatisfactory, due to the evident inconsistency in class distributions across the two graphs in Figure 7.

B.2 Do the same using the k-means algorithm, for k=3. For visualisation, you can pick the two features that were sufficient for classification in question A, and plot datapoints in these two dimensions, comparing actual classes and predicted cluster labels.

```
set.seed(1)
k <- 3
km <- kmeans(wine_training_noClass, centers = k, nstart = 50)
km_clusters <- km$cluster

# Compute the confusion matrix
cm_km <- table(Cluster = km_clusters, Class = actual_class)
print("Confusion matrix (k-means clusters vs true Class):")
```

```
## [1] "Confusion matrix (k-means clusters vs true Class):"
```

```
print(cm_km)
```

```
##         Class
## Cluster  1  2  3
##       1 11 17 20
##       2 38  1  0
##       3  0 41 14
```

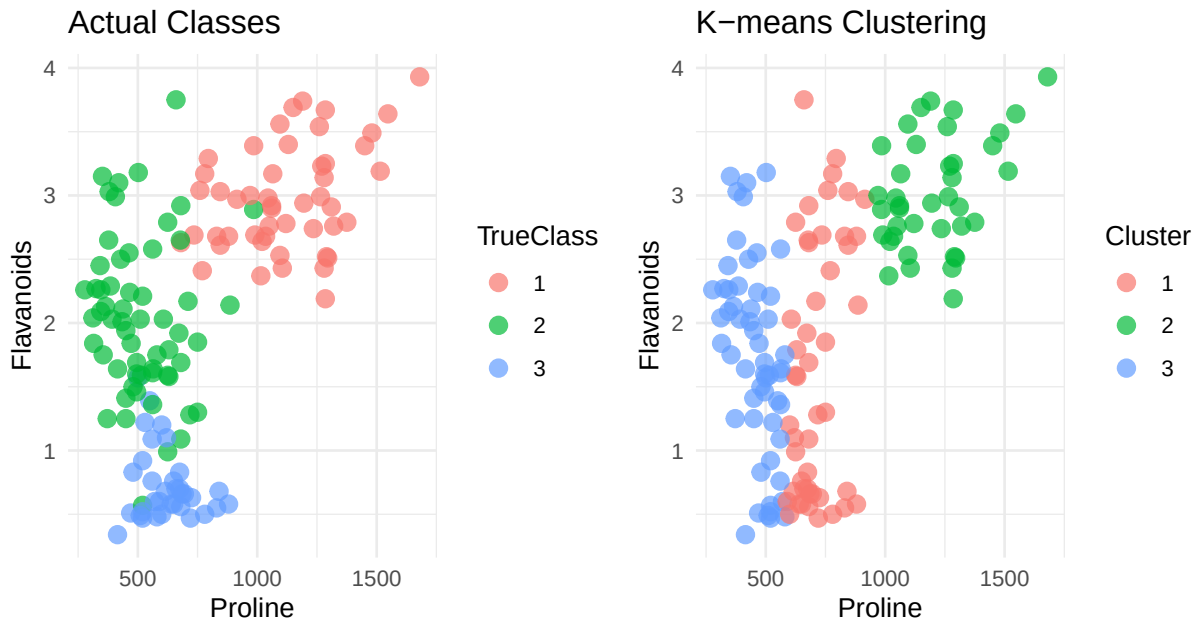
```
# Compute the purity of the confusion matrix
purity <- sum(apply(cm_km, 1, max)) / sum(cm_km)
cat("The purity of the confusion matrix using the unscaled training data is",
    round(purity,2))
```

```
## The purity of the confusion matrix using the unscaled training data is 0.7
```

The confusion matrix shows that the clustering results using k-means is still not good, due to the purity of 0.7.

```
# Visualise the actual classes compared to the k-means clustering results.
plot_data <- data.frame(
  Proline = wine_training$Proline,
  Flavanoids = wine_training$Flavanoids,
  TrueClass = as.factor(wine_training$Class),
  Cluster = as.factor(km_clusters)
)

# Ground Truth
p1 <- ggplot(plot_data, aes(x = Proline, y = Flavanoids, color = TrueClass)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(title = "Actual Classes",
       x = "Proline", y = "Flavanoids") +
  theme_minimal()

# The clustering results
p2 <- ggplot(plot_data, aes(x = Proline, y = Flavanoids, color = Cluster)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(title = "K-means Clustering",
       x = "Proline", y = "Flavanoids") +
  theme_minimal()

final_plot <- p1 + p2 + plot_annotation(
  title = "Figure 8. Comparison of Actual Classes vs K-means Clustering",
  theme = theme(plot.title = element_text(hjust = 0.5, size = 14, face = "bold"))
)

print(final_plot)
```

Figure 8. Comparison of Actual Classes vs K−means Clustering



A comparison between the k-means clustering results and the actual classes(Figure 8) reveals a clear mismatch on the left-hand side of the plot, where the distribution of two clusters does not align well with the corresponding true classes.

B.3 Normalise all numerical features using either z-score transformation or min-max normalisation, which will bring them into comparable orders of magnitude. Then repeat parts 1. and 2., and see whether your results improve.

B.3.1. Hierarchical clustering using normalized data

```
# Exclude the variable 'Class', and normalize the training data
scaled_wine_train = scale(wine_training_noClass)
dist_matrix_scale <- dist(scaled_wine_train)

# perform hierarchical clustering with normalized training data
hc_complete_norm <- hclust(dist_matrix_scale, method = "complete")

# Check the result of hierarchical clustering with the variable 'Class'.
hc_clusters_norm <- cutree(hc_complete_norm, k = 3)
actual_class <- as.factor(wine_training$Class)

# Compute the confusion matrix on training data set
cm_norm <- table(Cluster = hc_clusters_norm, Class = actual_class)
print(cm_norm)
```

```
##         Class
## Cluster  1  2  3
##       1  0 26 34
##       2 49 15  0
##       3  0 18  0
```

```r
# Compute the purity of the confusion matrix
purity <- sum(apply(cm_norm, 1, max)) / sum(cm_norm)
cat("The purity of the confusion matrix using the scaled training data is",
    round(purity,2))
```

```
## The purity of the confusion matrix using the scaled training data is 0.71
```

Given the unsatisfactory confusion matrix purity (0.71) obtained from hierarchical clustering with complete linkage on the scaled training data, we apply Ward's linkage while keeping all other parameters unchanged.

```r
# Change the complete linkage to ward linkage
# Perform hierarchical clustering with normalized training data
hc_complete_norm_ward <- hclust(dist_matrix_scale, method = "ward.D2")

# Check the result of hierarchical clustering with the variable 'Class'.
hc_clusters_norm_ward <- cutree(hc_complete_norm_ward, k = 3)
actual_class <- as.factor(wine_training$Class)

# Compute the confusion matrix on training data set
cm_norm_Ward <- table(Cluster = hc_clusters_norm_ward, Class = actual_class)
print(cm_norm_Ward)
```

```
##        Class
## Cluster  1  2  3
##       1  0  5 34
##       2 49  2  0
##       3  0 52  0
```

```r
# Compute the purity of the confusion matrix
purity <- sum(apply(cm_norm_Ward, 1, max)) / sum(cm_norm_Ward)
cat("The purity of the confusion matrix using the scaled training data is",
    round(purity,2))
```

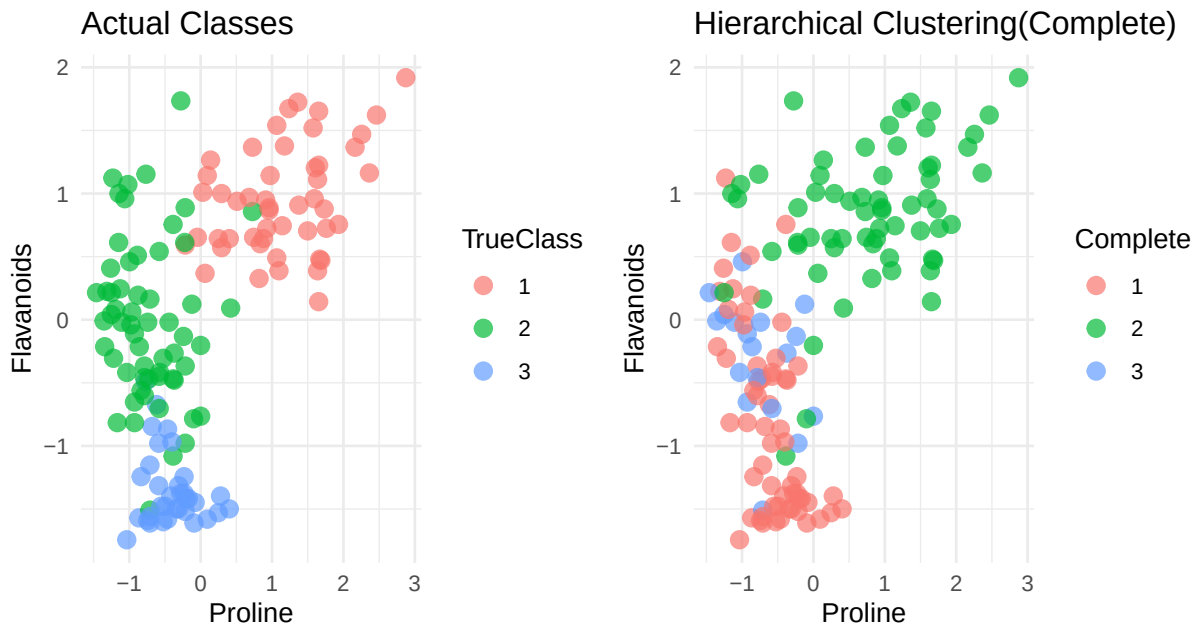
```
## The purity of the confusion matrix using the scaled training data is 0.95
```

When applying hierarchical clustering on the normalized training data, the purity improvement of the confusion matrix is negligible(from 0.68 to 0.71), suggesting that normalization only slightly enhances the alignment between the clusters and the true classes. Therefore,in hierarchical clustering, the advantages of data standardization are not necessarily significant. When we change the 'complete linkage' to 'ward linkage', the hierarchical clusters has much higher performance, with the purity of the confusion matrix increasing to 0.95.

```r
# Visualise the actual classes compared to the hierarchical clustering results(Normalized).
plot_data <- data.frame(
  Proline = scaled_wine_train[, "Proline"],
  Flavanoids = scaled_wine_train[, "Flavanoids"],
  TrueClass = as.factor(wine_training$Class),
  Complete = as.factor(hc_clusters_norm),
  Ward = as.factor(hc_clusters_norm_ward)
)

# Ground Truth
p1 <- ggplot(plot_data, aes(x = Proline, y = Flavanoids, color = TrueClass)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(title = "Actual Classes",
```

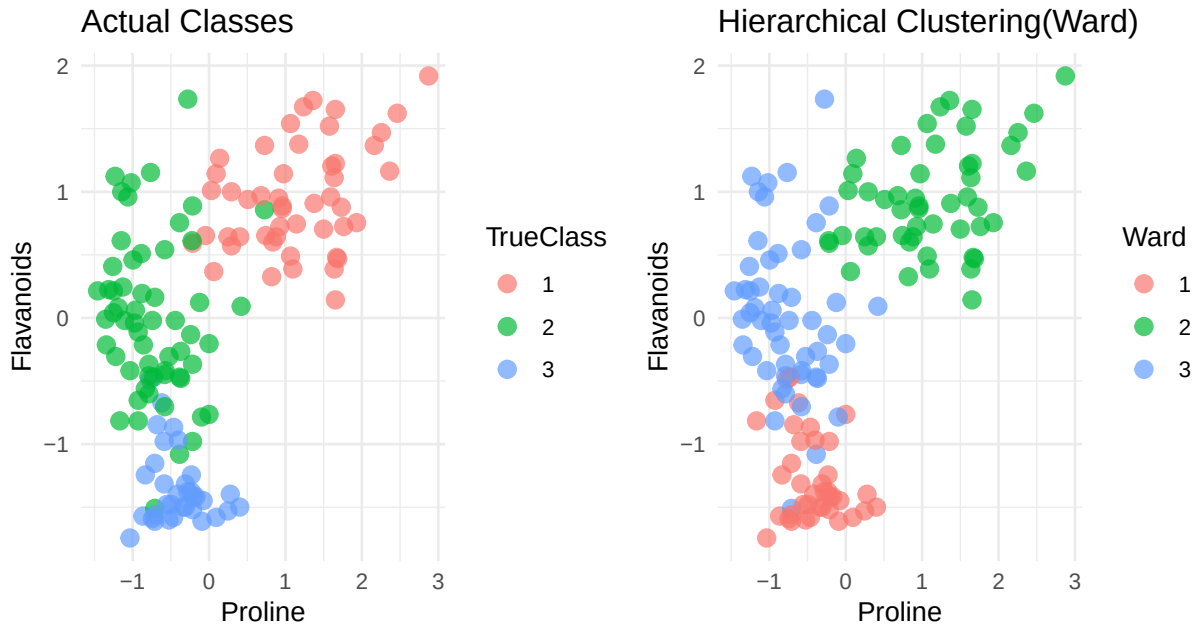
```
       x = "Proline", y = "Flavanoids") +
  theme_minimal()

# The clustering results using complete linkage
p2 <- ggplot(plot_data, aes(x = Proline, y = Flavanoids, color = Complete)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(title = "Hierarchical Clustering(Complete)",
       x = "Proline", y = "Flavanoids") +
  theme_minimal()

# The clustering results using ward linkage
p3 <- ggplot(plot_data, aes(x = Proline, y = Flavanoids, color = Ward)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(title = "Hierarchical Clustering(Ward)",
       x = "Proline", y = "Flavanoids") +
  theme_minimal()

final_plot12 <- p1 + p2 + plot_annotation(
  title = "Figure 9. Comparison of Actual Classes vs Hierarchical Clustering(Complete)",
  theme = theme(plot.title = element_text(hjust = 0.5, size = 12, face = "bold"))
)

final_plot13 <- p1 + p3 + plot_annotation(
  title = "Figure 10. Comparison of Actual Classes vs Hierarchical Clustering(Ward)",
  theme = theme(plot.title = element_text(hjust = 0.5, size = 12, face = "bold"))
)

print(final_plot12)
```

Figure 9. Comparison of Actual Classes vs Hierarchical Clustering(Complete)



```
print(final_plot13)
```

Figure 10. Comparison of Actual Classes vs Hierarchical Clustering(Ward)



The comparison plots further confirm that hierarchical clustering with complete linkage on the scaled data improves performance only slightly, with some class confusion still remaining (Figure 9), whereas hierarchical clustering with Ward's linkage on the scaled data produces clusters that closely match the actual classes (Figure 10).

B.3.2. K-means clustering based on standardized data.

```
set.seed(1)
k <- 3
km_norm <- kmeans(scaled_wine_train, centers = k, nstart = 50)
km_clusters_norm <- km_norm$cluster

# Compute the confusion matrix
cm_km_norm <- table(Cluster = km_clusters_norm, Class = actual_class)
print("Confusion matrix (kmeans clusters vs true Class):")
```

```
## [1] "Confusion matrix (kmeans clusters vs true Class):"
```

```
print(cm_km_norm)
```

```
##         Class
## Cluster  1  2  3
##       1  0 55  0
##       2 49  1  0
##       3  0  3 34
```

```
# Compute the purity of the confusion matrix
purity <- sum(apply(cm_km_norm, 1, max)) / sum(cm_km_norm)
cat("The purity of the confusion matrix using the normalised training data is",
```

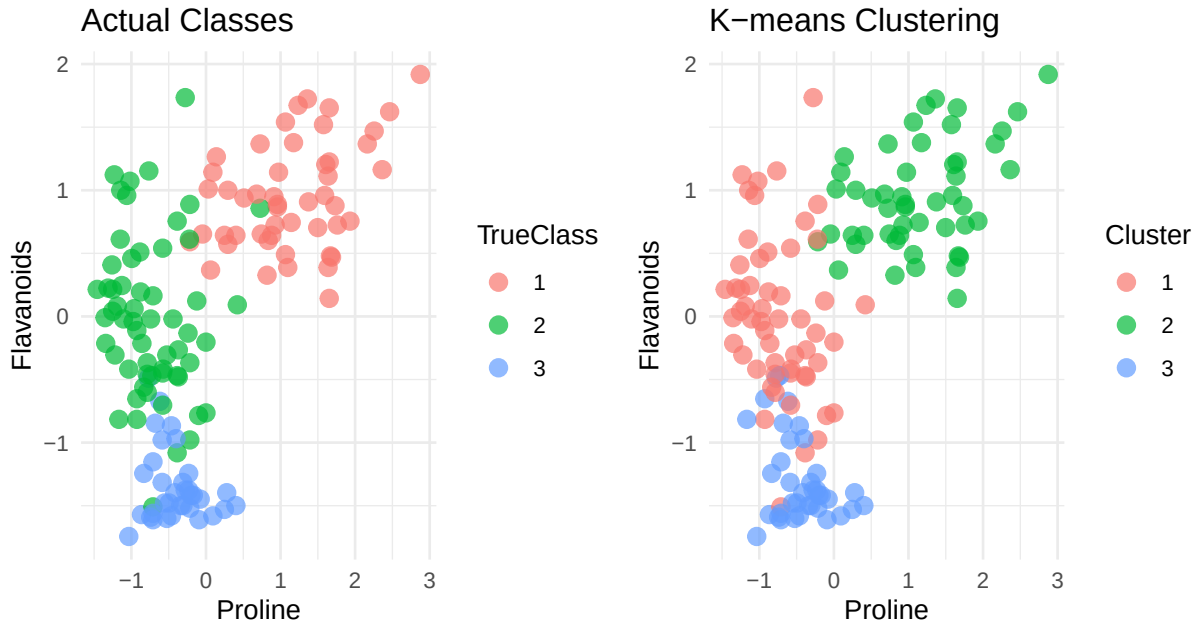
```
    round(purity,2))
```

```
## The purity of the confusion matrix using the normalised training data is 0.97
```

When applying k-means clustering on the normalized training data, the purity of the confusion matrix improved markedly from 0.70 to 0.97, suggesting that normalization greatly enhances the alignment between the clusters and the true classes. As a result, features with smaller numeric ranges (e.g., Flavanoids) contribute equally to the clustering, leading to a more balanced and accurate partition of the data.

```
# Visualize the actual classes compared to the k-means clustering results(Normalized).
plot_data <- data.frame(
  Proline = scaled_wine_train[, "Proline"],
  Flavanoids = scaled_wine_train[, "Flavanoids"],
  TrueClass = as.factor(wine_training$Class),
  Cluster = as.factor(km_clusters_norm)
)

# Ground Truth
p1 <- ggplot(plot_data, aes(x = Proline, y = Flavanoids, color = TrueClass)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(title = "Actual Classes",
       x = "Proline", y = "Flavanoids") +
  theme_minimal()

# The clustering results
p2 <- ggplot(plot_data, aes(x = Proline, y = Flavanoids, color = Cluster)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(title = "K-means Clustering",
       x = "Proline", y = "Flavanoids") +
  theme_minimal()

final_plot <- p1 + p2 + plot_annotation(
  title = "Figure 11. Comparison of Actual Classes vs K-means Clustering",
  theme = theme(plot.title = element_text(hjust = 0.5, size = 12, face = "bold"))
)

print(final_plot)
```

Figure 11. Comparison of Actual Classes vs K−means Clustering



The comparison plots (Figure 11) further confirm that, after normalization, the k-means clustering results are very close to the actual class labels, with only a few points remaining inconsistent.